

CENTRO UNIVERSITÁRIO UNIFECAP

Algorithmic Thinking & Introduction to Object-Oriented Programming

ORÇAMENTO DE ALUGUEL

Parte Teórica: Fluxograma e Estrutura Lógica

Eduardo Souza Mattos

R.A.: 35984

São Paulo

2026

Eduardo Souza Mattos

ORÇAMENTO DE ALUGUEL

Parte Teórica: Fluxograma e Estrutura Lógica

Documento referente à Parte Teórica (25% da nota) do trabalho avaliativo apresentado à disciplina de Algorithmic Thinking & Introduction to Object-Oriented Programming, do Centro Universitário UniFECAF, contendo o fluxograma da aplicação e a descrição da lógica algorítmica empregada.

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO	3
2 DESAFIO	3
3 LÓGICA DO ALGORITMO	5
3.1 Estruturas de decisão e repetição utilizadas	
3.2 Pseudocódigo da lógica principal	
3.3 Relação com a Programação Orientada a Objetos	
4 FLUXOGRAMA DA APLICAÇÃO	9
5 ETAPAS E SOFTWARES UTILIZADOS NA REPRODUÇÃO	10
6 CONSIDERAÇÕES FINAIS	10
REFERÊNCIAS	11

1 INTRODUÇÃO

Este documento corresponde à Parte Teórica do trabalho avaliativo da disciplina de Algorithmic Thinking & Introduction to Object-Oriented Programming, do Centro Universitário UniFECAF, referente ao desenvolvimento de uma aplicação de orçamento de aluguel para a empresa fictícia R.M. Conforme as orientações de entrega, esta parte corresponde a 25% da nota final e deve apresentar o fluxograma da aplicação e a descrição sucinta da lógica principal do programa, evidenciando como o pensamento algorítmico foi aplicado.

A Parte Prática (código-fonte, repositório GitHub) e o Vídeo Pitch são entregues separadamente, conforme os demais critérios de avaliação da disciplina.

A elaboração deste trabalho apoiou-se em materiais de apoio ao aprendizado da linguagem Python, incluindo a documentação oficial (PYTHON SOFTWARE FOUNDATION, 2026), tutoriais interativos voltados a iniciantes (LEARNPYTHON, 2026) e o conteúdo da disciplina disponibilizado no Ambiente Virtual de Aprendizagem (UNIFECAF, 2026), utilizados como base teórica para a estruturação da lógica apresentada nas seções seguintes.

2 DESAFIO

A empresa R.M solicitou o desenvolvimento de uma aplicação para automatizar a geração de orçamentos de locação de apartamentos, casas e estúdios. As regras de negócio que orientaram a construção do fluxograma e da lógica do algoritmo são:

- Apartamento: valor base de R\$ 700,00 (1 quarto); acréscimo de R\$ 200,00 para 2 quartos; acréscimo de R\$ 300,00 para garagem; desconto de 5% para clientes sem filhos.
- Casa: valor base de R\$ 900,00 (1 quarto); acréscimo de R\$ 250,00 para 2 quartos; acréscimo de R\$ 300,00 para garagem.
- Estúdio: valor base de R\$ 1.200,00; inclusão de 2 vagas de estacionamento por R\$ 250,00; cada vaga adicional custa R\$ 60,00.
- Contrato imobiliário no valor de R\$ 2.000,00, parcelável em até 5 vezes.
- Geração de um arquivo .csv contendo as 12 parcelas do orçamento anual.

3 LÓGICA DO ALGORITMO

A lógica do programa combina três elementos centrais do pensamento algorítmico estudados na disciplina (UNIFECAP, 2026): estruturas condicionais, laços de repetição e, na camada de implementação, os princípios da Programação Orientada a Objetos (herança e polimorfismo), descritos a seguir.

É importante destacar que a implementação não partiu de um código sem planejamento prévio. Duas bases orientaram diretamente a construção da lógica antes de qualquer linha de código ser escrita: primeiro, a especificação funcional detalhada do desafio (Seção 2), que já decompunha o problema em regras objetivas e mensuráveis (valores base, acréscimos, descontos, regras do contrato e formato de saída); segundo, a decisão arquitetural de modelar o domínio por meio de uma hierarquia de classes — uma classe abstrata Imóvel e as subclasses Apartamento, Casa e Estúdio —, definida antes da implementação da lógica interna de cada regra de precificação. Essa decisão de arquitetura funcionou como a estrutura orientadora ('espinha dorsal') da solução, sendo posteriormente complementada pela documentação em pseudocódigo e pelo fluxograma, descritos nas subseções seguintes.

3.1 Estruturas de Decisão e Repetição Utilizadas

A sintaxe e a semântica das estruturas condicionais (if/elif/else) e dos laços de repetição (while, for) empregados no código seguem o que é documentado no tutorial oficial da linguagem (PYTHON SOFTWARE FOUNDATION, 2026), aplicadas da seguinte forma no algoritmo:

- Estrutura condicional (se/senão): utilizada para identificar o tipo de imóvel escolhido e aplicar os acréscimos correspondentes (quartos, garagem, vagas de estacionamento) e o desconto para apartamentos sem crianças.
- Laço de repetição (enquanto): utilizado na validação das entradas do usuário, repetindo a solicitação até que um valor válido seja informado — evitando que o programa seja interrompido por uma entrada incorreta.
- Laço de repetição (para): utilizado para percorrer os 12 meses do orçamento anual, somando a parcela do contrato ao valor do aluguel apenas nos meses em que o contrato ainda está sendo pago.

3.2 Pseudocódigo da Lógica Principal

O Quadro 1 apresenta o pseudocódigo que resume a lógica principal da aplicação, da seleção do tipo de imóvel até a exportação do arquivo .csv. Comentários explicativos (precedidos por //) foram inseridos ao longo do pseudocódigo para esclarecer a finalidade de cada etapa e sua correspondência com as regras de negócio da empresa R.M.

```

INÍCIO
// Etapa 1: menu de seleção do tipo de imóvel
EXIBIR menu (Apartamento / Casa / Estúdio)
LER opção do usuário

SE opção = Apartamento ENTÃO
    // Regras específicas do Apartamento
    LER quartos, garagem, tem_crianças
    valor <- 700.00 // valor base (1 quarto)
    SE quartos >= 2 ENTÃO valor <- valor + 200.00 // 2º quarto
    SE garagem = verdadeiro ENTÃO valor <- valor + 300.00 // garagem
    SE tem_crianças = falso ENTÃO
        valor <- valor - (valor * 5%) // desconto sem filhos

SENÃO SE opção = Casa ENTÃO
    // Regras específicas da Casa (sem desconto por filhos)
    LER quartos, garagem
    valor <- 900.00 // valor base (1 quarto)
    SE quartos >= 2 ENTÃO valor <- valor + 250.00 // 2º quarto
    SE garagem = verdadeiro ENTÃO valor <- valor + 300.00 // garagem

SENÃO (Estúdio)
    // Regras específicas do Estúdio (vagas de estacionamento)
    LER vagas_estacionamento
    valor <- 1200.00 // valor base do estúdio
    SE vagas_estacionamento > 0 ENTÃO
        valor <- valor + 250.00 // pacote com as 2 primeiras vagas
        vagas_extras <- MAX(0, vagas_estacionamento - 2) // vagas extras
        valor <- valor + (vagas_extras * 60.00) // R$60 por vaga extra

// Etapa 2: parcelamento do contrato (comum aos três tipos)
LER num_parcelas_contrato (1 a 5)
parcela_contrato <- 2000.00 / num_parcelas_contrato

// Etapa 3: geração do orçamento anual (laço PARA)
PARA mes DE 1 ATÉ 12 FAÇA
    SE mes <= num_parcelas_contrato ENTÃO
        contrato_no_mes <- parcela_contrato // ainda pagando
    SENÃO
        contrato_no_mes <- 0.00 // contrato já quitado
    total_mes <- valor + contrato_no_mes
    REGISTRAR (mes, valor, contrato_no_mes, total_mes)

// Etapa 4: apresentação e exportação do resultado
EXIBIR as 12 parcelas na tela
EXPORTAR as 12 parcelas para arquivo .csv
FIM

```

Quadro 1 – Pseudocódigo da lógica principal do algoritmo

3.3 Relação com a Programação Orientada a Objetos

Conforme descrito por DevMedia (2026), a orientação a objetos em Python permite organizar um sistema em classes que representam entidades do mundo real, utilizando herança para reaproveitar comportamentos comuns. Na implementação deste projeto, essa lógica foi organizada por meio de uma classe abstrata `Imovel`, da qual herdam as classes `Apartamento`, `Casa` e `Estudio`. Cada subclasse sobrescreve o método `calcular_valor_mensal()`, aplicando suas próprias regras — um exemplo de polimorfismo, em que o mesmo método é chamado de forma idêntica pelo restante do programa, mas se comporta de maneira diferente conforme o tipo de objeto instanciado. A classe `Orcamento`, por sua vez, recebe qualquer instância de `Imovel` e é responsável por gerar as 12 parcelas mensais e exportá-las para um arquivo `.csv`, sem precisar conhecer as regras internas de cada tipo de imóvel.

Essa separação de responsabilidades exemplifica o encapsulamento, princípio também reforçado nos exercícios práticos de orientação a objetos disponibilizados por Google (2026): cada classe conhece apenas o que é necessário para calcular seu próprio valor, enquanto a orquestração do orçamento (parcelamento do contrato, geração das 12 parcelas e exportação) permanece isolada na classe `Orcamento`, tornando o código mais organizado, reutilizável e fácil de manter.

4 FLUXOGRAMA DA APLICAÇÃO

Uma vez definida a lógica descrita na Seção 3, essa lógica foi traduzida visualmente no fluxograma apresentado na Figura 1, documentando o pensamento algorítmico aplicado na construção da aplicação de orçamento de aluguel. O processo se inicia com a seleção do tipo de imóvel, segue por uma estrutura de decisão que direciona o fluxo para a coleta de dados específicos de cada tipo (apartamento, casa ou estúdio), calcula o valor mensal aplicando as regras de acréscimo e desconto, calcula a parcela do contrato imobiliário e, por fim, gera e exporta o orçamento das 12 parcelas em um arquivo .csv.

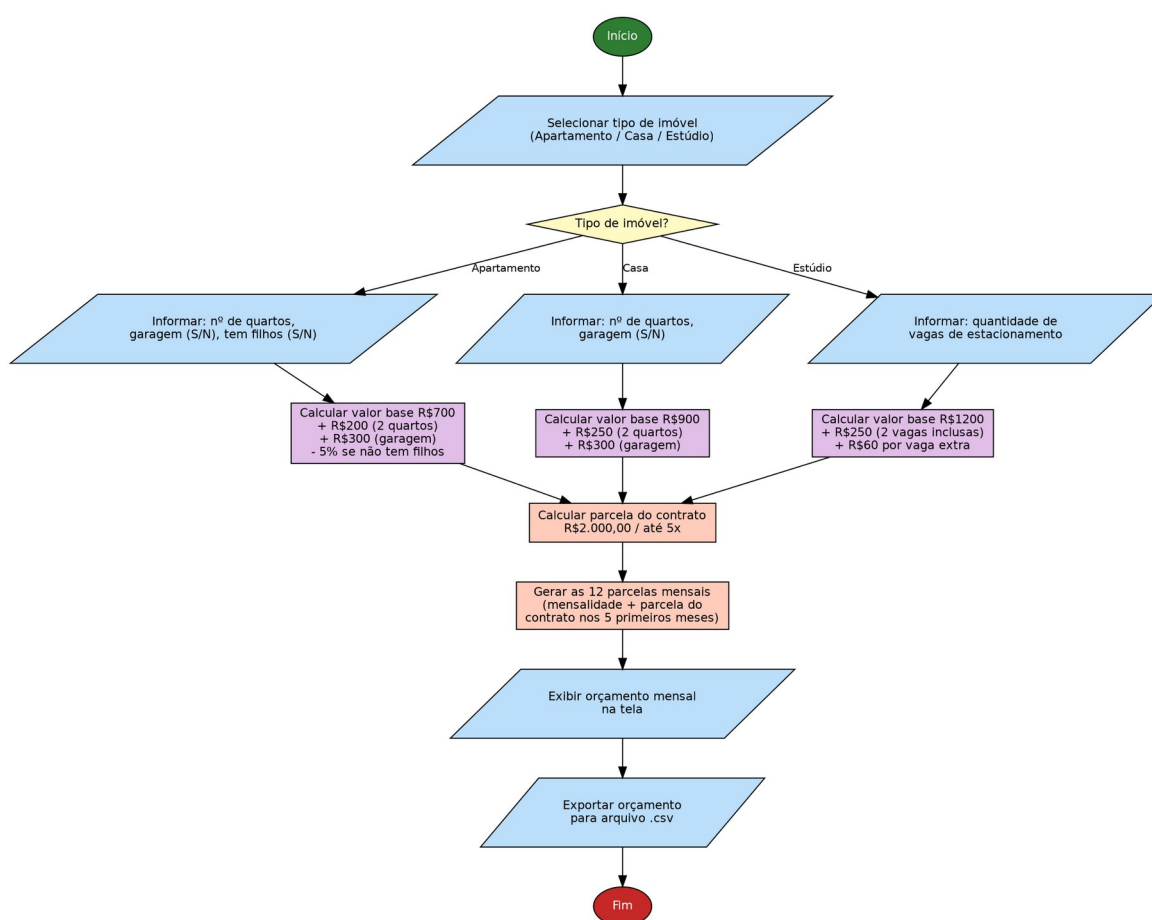


Figura 1 – Fluxograma da aplicação de Orçamento de Aluguel
Fonte: Elaborado pelo autor com a biblioteca Graphviz (2026).

5 ETAPAS E SOFTWARES UTILIZADOS NA REPRODUÇÃO

A reprodução deste trabalho segue três etapas, na seguinte ordem: primeiro o código-fonte, depois a geração do orçamento (.csv) a partir dele e, por fim, a geração do fluxograma que documenta essa lógica. O uso de um script Python auxiliar para gerar automaticamente o fluxograma, em vez de desenhá-lo manualmente, segue a abordagem prática de automatização de tarefas com Python incentivada por Xavier (2026). Cada etapa emprega softwares específicos, descritos a seguir.

- Etapa 1 — Código-fonte (`orcamento_aluguel.py`): escrito e editado utilizando Python 3 e o editor Visual Studio Code, no sistema operacional Pop!_OS.
- Etapa 2 — Geração do `orcamento_aluguel.csv`: obtida executando o código-fonte da Etapa 1 (`python3 orcamento_aluguel.py`), que gera automaticamente o arquivo .csv com as 12 parcelas; opcionalmente, o LibreOffice Calc foi utilizado para conferir visualmente o conteúdo exportado.
- Etapa 3 — Geração do `fluxograma_orcamento.png` (Figura 1): obtida a partir de um script auxiliar (`flowchart.py`), utilizando a biblioteca Graphviz e o software Graphviz instalado no sistema operacional.

6 CONSIDERAÇÕES FINAIS

A lógica do programa foi orientada, desde o início, pela especificação funcional do desafio e pela arquitetura de classes definida previamente (Seção 3), e implementada diretamente em Python por meio das classes ali descritas. Essa lógica foi posteriormente documentada em pseudocódigo (Quadro 1) e representada visualmente no fluxograma (Figura 1), gerado com auxílio da biblioteca Graphviz. Essa documentação complementar se mostrou fundamental para comunicar de forma clara o pensamento algorítmico empregado, especialmente diante das três regras de precificação distintas exigidas pela empresa R.M. A tradução dessa lógica em pseudocódigo e em fluxograma evidencia, de forma didática, a aplicação prática dos conceitos de estruturas condicionais, laços de repetição e Programação Orientada a Objetos estudados na disciplina.

REFERÊNCIAS

DEVMEDIA. Guia completo de Python. 2026. Disponível em: <https://www.devmedia.com.br/curso/python>. Acesso em: 29 jul. 2026.

GOOGLE. Google for Developers: Python. 2026. Disponível em: <https://developers.google.com/edu/python?hl=pt-br>. Acesso em: 29 jul. 2026.

LEARNPYTHON. Learn Python. 2026. Disponível em: <https://learnpython.org/>. Acesso em: 29 jul. 2026.

PYTHON SOFTWARE FOUNDATION. The Python Tutorial. 2026. Disponível em: <https://www.python.org/doc/>. Acesso em: 29 jul. 2026.

UNIFECAP. Algorithmic Thinking & Introduction to Object-Oriented Programming: Unidade I – Estruturando Dados. São Paulo: Centro Universitário UniFECAP, 2026. Material de disciplina disponibilizado no Ambiente Virtual de Aprendizagem (AVA).

XAVIER, Daniel. Aprendendo do início com Daniel. 2026. Disponível em: <https://www.youtube.com/@aprendendo.doinicio>. Acesso em: 29 jul. 2026.